Fibonacci heaps

Part V Advanced Data Structures

Scope

Deferred consolidation gives meld and decrease-key low amortized costs. Root/child rings, loss marks, cascading cuts, and degree collisions must fit one potential argument. A Fibonacci subtree bound limits the consolidation array.

Prerequisites

Mergeable heaps, amortized analysis, and cut marks.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter route

Learning objectives

- Represent a relaxed heap with root and child lists.
- Amortize meld, insertion, and decrease-key.
- Prove consolidation cost using a degree bound.
- 19.1 Structure of Fibonacci heaps
- 19.2 Mergeable-heap operations
- 19.3 Decreasing a key and deleting a node
- 19.4 Bounding the maximum degree

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Structure of Fibonacci heaps

A Fibonacci heap is a collection of min-heap-ordered trees. Every nonroot x satisfies parent.key≤x.key. The root list contains all roots, H.min points to a minimum-key root, and H.n counts all nodes. Decrease-key and erase accept valid node handles; their bounds exclude searching for an arbitrary key.

Each node stores key, degree, parent, child, left, right, and mark. Degree is its current child count; child points to one child. Siblings form circular doubly linked lists. We clear every root's mark. A marked nonroot has lost one child since it was last linked to its current parent.

Keys may repeat while handle identities differ. Deletion invalidates its handle. Meld consumes ownership of both input heaps; callers cannot keep mutating the same nodes through separate inputs. These premises define the objects covered by pointer and cost claims.

Constant-time ring pointer operations

Before removing x, save next=x.right. For a singleton set the ring head to NIL; otherwise reconnect x.left.right=x.right and x.right.left=x.left, updating head=next if needed. Finally set x.left=x.right=x and update its container's head/degree.

To concatenate nonempty rings A,B, choose a∈A,b∈B and save an=a.right,bp=b.left, then write

```
a.right=b; b.left=a
bp.right=an; an.left=bp
```

Only two adjacency edges change; no complete traversal is needed. An empty ring returns the other. Before adding x to roots, detach it from its child ring and decrement the old parent's degree, then set x.parent=NIL and mark=false. Overwriting x.right before traversing its original ring can lose the successor.

Mergeable-heap operations

MAKE-HEAP creates an empty root list, min=NIL, n=0. INSERT creates a degree-zero singleton ring with parent/child=NIL and mark=false, joins it to roots, updates min by comparison, and increments n. MINIMUM returns min, or NIL if empty.

MELD concatenates root rings, chooses the smaller input minimum, and adds node counts. It neither links trees nor scans nodes. Since each input already tracks its smallest root, one comparison suffices. These operations have O(1) actual cost.

Use $\Psi=C(t+2m)$, for a sufficiently large fixed C, root count t, and marked-node count m. Across multiple heaps, sum potentials. It starts at0 and remains nonnegative. INSERT adds one root and C potential, hence O(1) amortized. MELD preserves total roots and marks, giving $\Delta\Psi=0$ and O(1) amortized cost.

Complete extract-min order

```
extract-min(H):
    z=H.min
    if z==NIL: return NIL
    while z.child ring is nonempty:
        detach one child x from z
        x.parent=NIL; x.mark=false
        append x to root list
    remove z from root list; H.n--
    if H.n==0: H.min=NIL
    else: consolidate roots by degree; recompute H.min
    invalidate z's heap handle; return z
```

Children become roots and retain internal heap order. Other roots were already no smaller than z. Recompute min among the surviving consolidated roots. Do not leave min pointing to old z or remove z's entire subtree.

Consolidation collision loop

```
A[0..D] = NIL
while old root list is nonempty:
    x=detach one root; d=x.degree
    while A[d]!=NIL:
        y=A[d]; A[d]=NIL
        if y.key<x.key: swap(x,y)
        make y a child of x; y.parent=x; y.mark=false
        x.degree++; d=x.degree
    A[d]=x
rebuild root list from nonempty A slots
H.min = smallest-key root
```

Each link combines equal-degree roots under the smaller key, preserving heap order. After incrementing d, check A again. The result has at most one root per degree. D must be at least floor(log_ϕ n), or the array can grow on demand. floor($\log_2 n$) alone need not fit legal degrees.

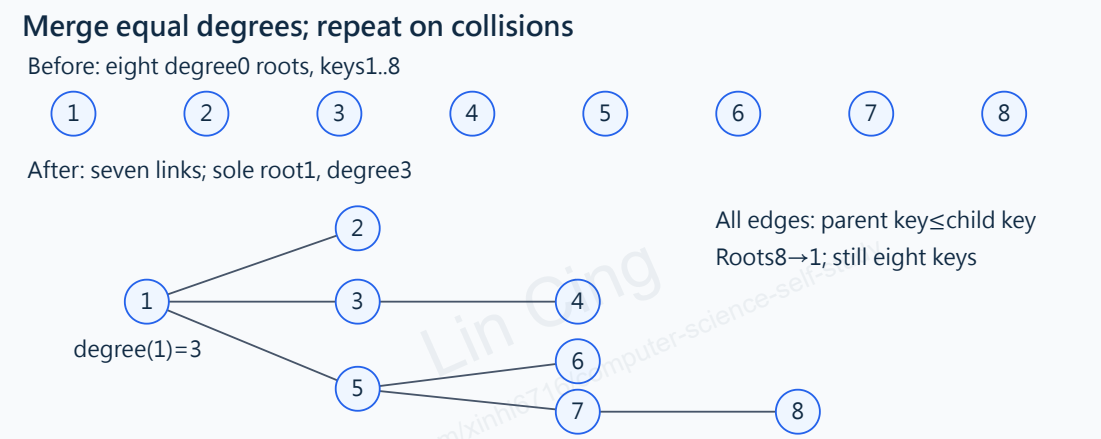
Consuming the old ring directly needs O(log n) auxiliary array space. A snapshot used to avoid mutation during traversal may contain Θ(n) roots, so its complete auxiliary space cannot still be called O(log n).

Numerical consolidation of eight roots

Insert0..8 and extract0, leaving singleton roots1..8. Process them in that order.

Root read	Links this round	Remaining A, degree:key
1	None	0:1
2	2 under1	1:1
3	None	0:3,1:1
4	4 under3;3 under1	2:1
5	None	0:5,2:1
6	6 under5	1:5,2:1
7	None	0:7,1:5,2:1
8	8 under7;7 under5;5 under1	3:1

Numerical consolidation of eight roots (continued)



Seven links reduce eight roots to one and preserve keys1..8. The tree need not be complete; every parent-child heap comparison and actual degree establish correctness.

Decreasing keys and cascading cuts

```
decrease-key(H,x,new):
    require new<=x.key and x is a valid handle
    x.key=new; p=x.parent
    if p!=NIL and x.key<p.key:
        cut x from p to root list
        cascading-cut(p)
    if x.key<H.min.key: H.min=x
cascading-cut(y):
    p=y.parent
    if p==NIL: return
    if y.mark==false: y.mark=true; return
    cut y from p to root list
    cascading-cut(p)
```

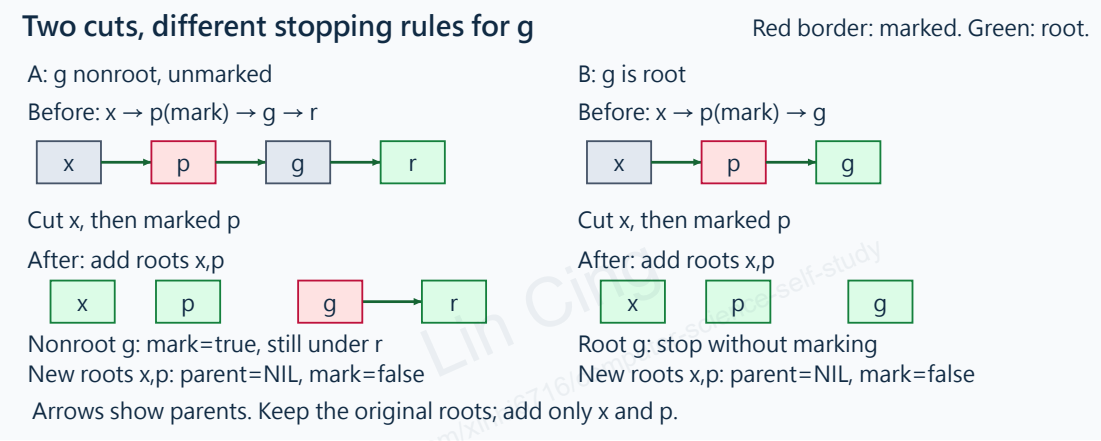
Cut clears the moved node's mark, decrements its old parent's degree, and preserves both ring directions. Lowering x can violate only its parent edge; its child comparisons become easier. Cutting the violated edge restores heap order. Marks separately protect the degree bound, rather than locating the minimum.

The two cascade stopping branches

If p was already marked, losing second child x cuts p to roots too. Inspect its old parent g next. If g is a nonroot and unmarked, mark it and stop. If g is root, stop without marking it for this loss.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The two cascade stopping branches (continued)



The root list **adds** x,p while retaining existing roots. Both have parent=NIL and mark=false. A nonroot g stays under its old parent and changes only its mark. If g was already marked, continue cutting rather than applying the unmarked branch.

Actual nodes from a legal history

Insert0..64 and extract0 to produce a consolidated64-node heap. Node IDs initially equal keys and stay fixed after decreases.

Branch	First operation	Second operation	Final state
g root	Lower id6 to−10;cut6,mark p=id5	Lower id7 to−11;cut7 then5	roots={1,6,7,5};g=id1 unmarked
g nonroot	Lower id14 to−10;cut14,mark p=id13	Lower id15 to−11;cut15 then13	roots={1,14,15,13};g=id9 marked,still under id1

After the first operation, both have t=2,m=1 and t+2m=4. After the second, the root branch has t=4,m=0 and scale4; the nonroot branch t=4,m=1 and scale6. These parents arise from legal links and cuts, rather than assuming an arbitrary shape is reachable.

Complete decrease-key potential accounting

Without a cut, actual cost and potential change are O(1). If c≥1 nodes are cut, roots increase by c. Apart from the first x cut because of its key, all c−1 cascading nodes were marked and lose their marks. At most one final unmarked nonroot gains a mark.

Thus Δm≤−(c−1)+1=2−c and

$$\Delta(t+2m)\le c+2(2-c)=4-c.$$

If actual cost≤ac+b, fix C≥a in the same Ψ=C(t+2m). Then

$$\hat{c}\le ac+b+C(4-c)\le b+4C=O(1).$$

Scaling explicitly pays the actual pointer-operation constants. One decrease-key can traverse a long ancestor chain; the constant is an amortized bound for operation sequences.

Deletion cost and a special minimum key

Abstract CLRS DELETE(x) first decreases x to a special $-\infty$ strictly below every ordinary key, then extracts minimum. Its amortized cost is $O(1)+O(\log n)=O(\log n)$, not $O(1)$.

That $-\infty$ is an extra key-domain element, not necessarily an integer type's minimum. If an ordinary node already has that value, ties can cause extract-min to remove another handle.

Alternatively erase the handle without changing its key. If x is nonroot, cut x and cascade from its old parent. Remove that specific root x, promote its children, consolidate, and recompute min. The cut stage is $O(1)$ amortized and root removal $O(\log n)$, not constant-time deletion that omits consolidation.

Bounding maximum degree

For fixed x, order its current k children $y_1..y_k$ by when they were linked to x. y_1 has degree ≥ 0. When y_i joined for $i \geq 2$, the earlier $i-1$ children that still survive were already present, so x then had degree $\geq i-1$.

Links combine equal-degree roots, so y_i then also had degree $\geq i-1$. While still a child of x, y_i can have lost at most one child since then; the second loss would have cut it. Therefore

$\text{degree}(y_i) \geq i-2$.

This uses insertion order of **currently surviving** children. It assumes neither that x never lost a child nor that all current child degrees are equal.

Fibonacci lower bound on subtree size

Let F0=0,F1=1,F_(j+2)=F_(j+1)+F_j. Prove size(x)≥F_(degree(x)+2) by strong induction on subtree size N.

N=1 gives degree0 and F2=1. For N>1 every child subtree is smaller, so apply induction to its actual degree; no assumption that a child's degree is below its parent's is needed. Child degree bounds and monotonicity give, for k≥1,

`size(x)≥1+1+Σ_(i=2)^k F_i=F_(k+2).`

The identity follows from Σ_(i=0)^kF_i=F_(k+2)−1 with F0=0,F1=1. At k=1 the empty sum gives2=F3.

Set ϕ=(1+√5)/2. F2=1=ϕ⁰,F3=2≥ϕ. Adding previous bounds and using $\phi^2=\phi+1$ inductively gives F_(k+2)≥ϕ^k. Thus n≥size(x)≥ϕ^k and k≤log_ϕ n. This bounds degree, not tree height.

Extract-min's total potential account

Before removal there are t roots and at most D children under the minimum. Removal/promotion leaves at most t−1+D roots. Each link reduces root count by1, so actual consolidation work is O(t+D), not O(tD) despite the nested collision loop.

Afterward distinct degrees give at most D+1 roots and marks do not increase, so Δ(t+2m)≤D+1−t. With the same sufficiently large C,

`ĉ≤a(t+D)+b+C(D+1−t)=O(D+1).`

The degree proof gives D=O(log(n+1)). Extract-min and delete are O(log(n+1)) amortized. Minimum,insert,meld,decrease-key are O(1) amortized; the first three also have O(1) actual cost. All conclusions telescope the same nonnegative potential. Finite operation tests check diagrams,numerical cases,and implementation states rather than replacing these proofs.

Sources

Bibliography

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter19.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study